Machine Learning — Assignment 2

M.Tech (AIML) · Work Integrated Learning Programmes Division, BITS Pilani
Course: Machine Learning (S2-25_AIMLCZG565)
Student: Harikumar T · ID: 2025AC05989
Topic: Telecom Customer Churn — Classification and Model Comparison

---

1. GitHub Repository Link

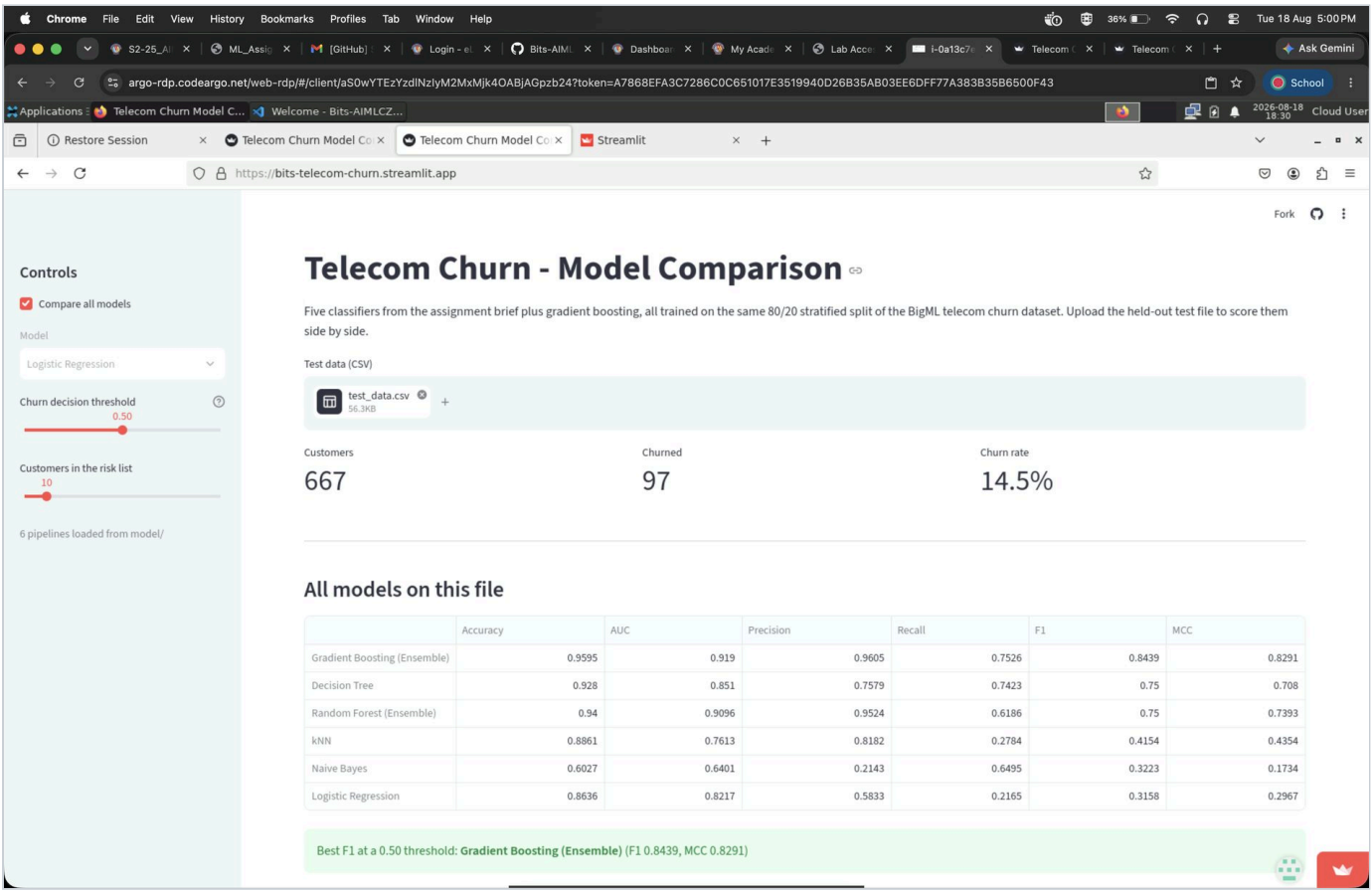
> https://github.com/2025ac05989-bits/Bits-AIMLCZG565-Semester-1-Assignment-2

The repository contains the complete source code (`app.py`, `churn_features.py`, `model/churn_modelling.ipynb`), `requirements.txt`, the README, the test data used for the experiments (`test_data.csv`), and the six saved model pipelines under `model/`.

2. Live Streamlit App Link

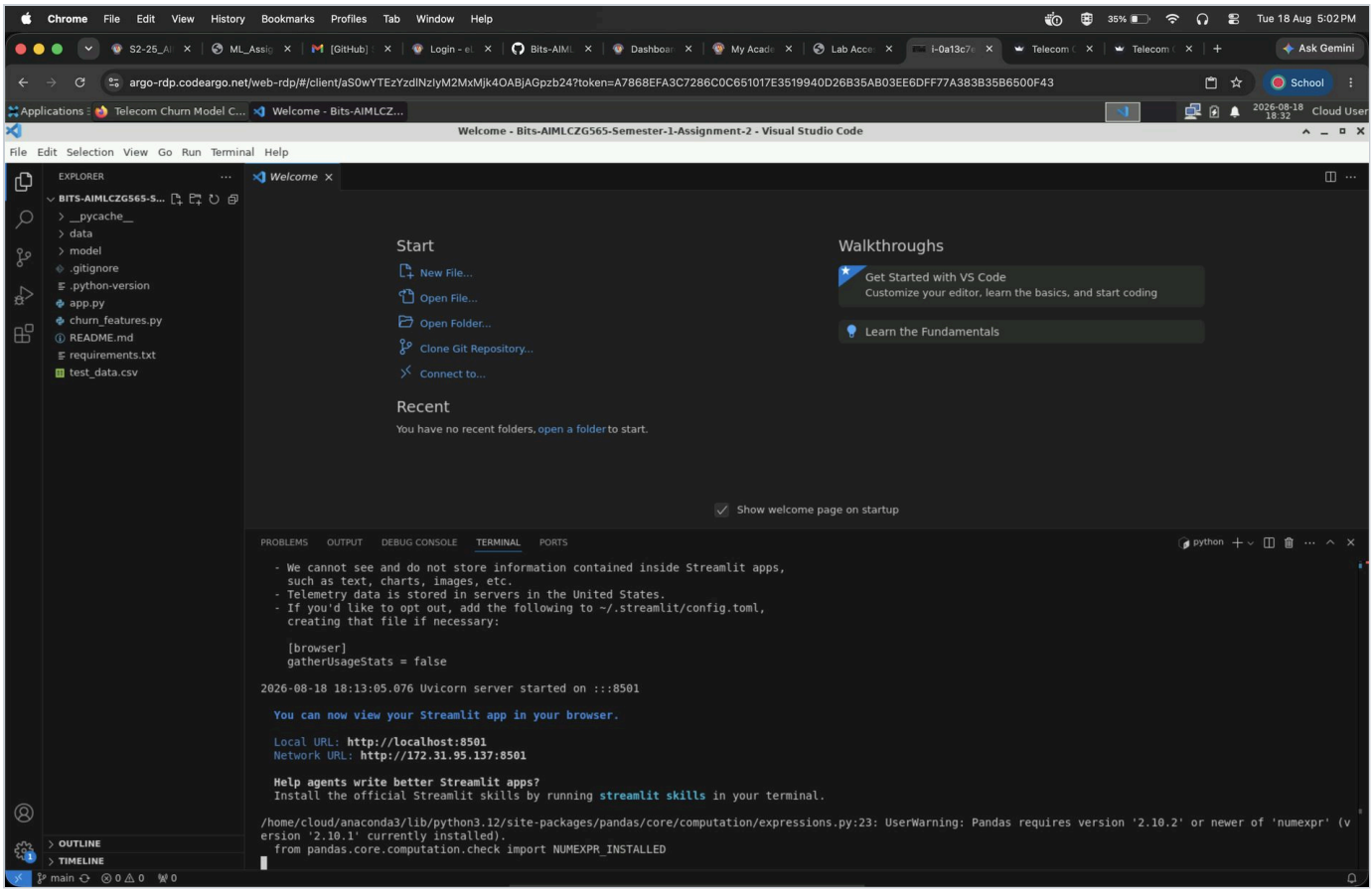
> https://bits-telecom-churn.streamlit.app/

Deployed on Streamlit Community Cloud from the `main` branch of the repository above.

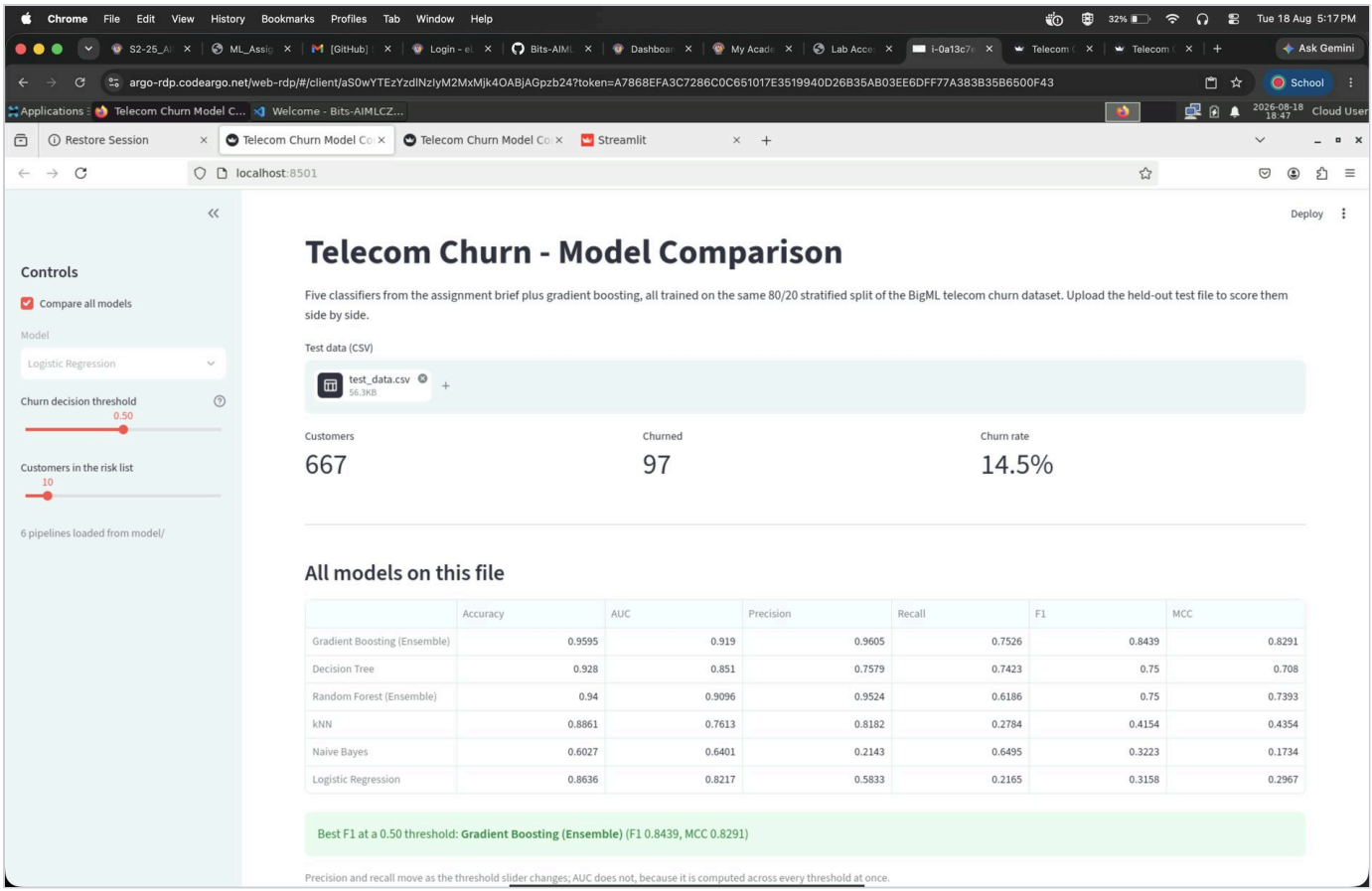


The deployed application, with `test_data.csv` uploaded and all six models compared on the held-out test split.

3. Screenshot of Assignment Execution on BITS Virtual Lab



The repository cloned into the BITS Virtual Lab, with the Streamlit server running from the integrated terminal (Local URL: http://localhost:8501).



The application running locally in the Virtual Lab at `localhost:8501`, scoring the uploaded `test_data.csv` (667 customers, 97 churned, 14.5% churn rate). The metrics match the comparison table in the README.

4. README.md Content

Telecom Customer Churn - Classification and Model Comparison

Assignment 2, Machine Learning (S2-25_AIMLCZG565), M.Tech AIML.

Problem statement

A telecom operator loses revenue every time a subscriber leaves, and winning a customer back costs considerably more than keeping one. The operator therefore wants to know, from the account and usage record it already holds, which subscribers are about to leave - early enough to act on it.

That makes this a supervised binary classification problem. Given 19 attributes describing a customer's plan, call volumes, billing and support history, predict the `Churn` flag: `True` if the customer left, `False` if they stayed. Because only about one customer in seven actually churns, the interesting question is not overall accuracy but how many of that minority a model can find without drowning the retention team in false alarms - so the comparison below is read through recall, F1 and MCC rather than accuracy alone.

Dataset description

The BigML telecom churn extract, published on Kaggle as "Churn in Telecom's dataset". It ships as two CSVs (`churn-bigml-train.csv`, `churn-bigml-test.csv`), both kept in `data/`. I stacked them back together and made my own stratified split, so the split is reproducible from the notebook alone.

Property	Value
Instances	3,333 customers
Features	19 predictors + 1 target
Target	`Churn` - binary (`True` / `False`)
Class balance	2,850 retained (85.51%) / 483 churned (14.49%)
Missing values	None in any of the 20 columns
Train / test	2,666 / 667, stratified 80/20, `random_state=5989`

Feature breakdown, and how each group is handled in the pipeline:

Group	Columns	Treatment
Categorical	`State` (51 levels), `Area code` (3 levels)	One-hot encoded
Binary	`International plan`, `Voice mail plan` (Yes/No)	Mapped to 0/1
Numeric	15 columns - account length, vmail count, day/evening/night/international minutes, calls and charges, customer service calls	Standardised

Two things about this dataset are worth recording, because both changed how I read the results:

1. `Area code` **is not a number.** It is stored as an integer but only ever holds 408, 415 or 510. It is a region label, so it is encoded rather than scaled.
2. **The billing columns are redundant.** Each `Total ... charge` column is exactly its matching `Total ... minutes` column times a flat tariff - 17.0c per day minute, 8.5c evening, 4.5c night, 27.0c international - and the correlations are 1.0 to six decimal places. Four of the fifteen numeric predictors therefore add no new information. I kept them in, since feature selection was not part of the brief, but this directly predicts which model should suffer: Naive Bayes assumes conditional independence, and here four pairs are perfectly dependent by construction.

The strongest single predictor is `Customer service calls`. Churn sits between 10% and 13% for customers making up to three support calls, then jumps above 45% from the fourth call onward. It is a step rather than a gradient, which favours models that can split on a threshold. Customers on the international plan churn at 42.4% against 11.5% for everyone else.

GitHub repository

https://github.com/2025ac05989-bits/Bits-AIMLCZG565-Semester-1-Assignment-2

Live Streamlit app

https://bits-telecom-churn.streamlit.app/

Models used

All six models sit behind one shared `ColumnTransformer` wrapped in a scikit-learn `Pipeline`, so the scaling and encoding are fitted on the training fold only and every model sees identical inputs. All are left at library defaults - tuning only some of them would make the comparison meaningless. The brief lists five models; I added gradient boosting as a sixth so the random forest has a second ensemble to be measured against.

Metrics are computed on the 667-customer held-out split at the default 0.50 threshold.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.8636	0.8217	0.5833	0.2165	0.3158	0.2967
Decision Tree	0.9280	0.8510	0.7579	0.7423	0.7500	0.7080
kNN	0.8861	0.7613	0.8182	0.2784	0.4154	0.4354
Naive Bayes	0.6027	0.6401	0.2143	0.6495	0.3223	0.1734
Random Forest (Ensemble)	0.9400	0.9096	0.9524	0.6186	0.7500	0.7393
Gradient Boosting (Ensemble)	0.9595	0.9190	0.9605	0.7526	0.8439	0.8291

For reference, always predicting "no churn" scores **0.8546 accuracy** on this test set. Three of the six models fail to beat that baseline by a meaningful margin, which is the clearest argument for not reading the accuracy column on its own.

Observations on each model

ML Model Name	Observation about model performance
Logistic Regression	Accuracy of 0.8636 is barely above the 0.8546 do-nothing baseline, and recall of 0.2165 means it finds only about one churner in five. The AUC of 0.8217 is the interesting part: the model ranks customers by risk perfectly sensibly, so the failure is the 0.50 cut-off, not the ordering. A linear boundary cannot represent the step at four support calls, so it spreads that signal thinly across the coefficients. Lowering the threshold makes it genuinely usable, which is why the app has a threshold slider.
Decision Tree	Jumps to 0.9280 accuracy and F1 of 0.7500, with the most balanced precision/recall pair of any model here (0.7579 / 0.7423). It splits directly on `Customer service calls >= 4` and the international-plan flag, which is exactly the shape of the signal in this data. Precision is the weakest of the three tree-based models because the tree is unpruned and some leaves are fitted to noise.
kNN	Accuracy of 0.8861 looks respectable until recall of 0.2784 is read alongside it - it misses nearly three quarters of the churners, and its AUC of 0.7613 is the worst in the table. One-hot encoding 51 states pushes the feature space to roughly 70 dimensions, where Euclidean distances lose their meaning, and with an 85/15 split the nearest neighbours of a churner are usually retained customers who outvote it.
Naive Bayes	The only model that falls below the do-nothing baseline, at 0.6027 accuracy and an MCC of 0.1734. It behaves in the opposite way to the others: recall of 0.6495 is higher than logistic regression, kNN and even the random forest, but precision of 0.2143 means roughly four of every five churn alerts are wrong. The cause is visible in the data - conditional independence is violated outright by the four charge/minutes pairs, so the same evidence is counted twice and probabilities get pushed to extremes.
Random Forest (Ensemble)	The strongest of the five prescribed models: 0.9400 accuracy, 0.9096 AUC, and precision of 0.9524, meaning almost every customer it flags genuinely did churn. Recall of 0.6186 is its weakness, and notably lower than the single decision tree - averaging over many bootstrapped trees smooths away the minority-class splits that an individual tree commits to, so bagging buys precision at recall's expense.
Gradient Boosting (Ensemble)	Best on five of the six metrics: 0.9595 accuracy, 0.8439 F1, 0.8291 MCC, 0.9190 AUC. Boosting fits each tree to the errors the previous ones made, so the misclassified churners get progressively more weight - which is precisely the failure mode the bagged forest has on this imbalanced data. It recovers the random forest's precision (0.9605) while lifting recall from 0.6186 to 0.7526.
Overall winner for this dataset	**Gradient Boosting**, on F1 (0.8439) and MCC (0.8291) with the highest accuracy and AUC as well. Restricting to the five models named in the brief, the winner is **Random Forest** - it ties the decision tree on F1 at 0.7500 but wins the tie on accuracy (0.9400 vs 0.9280), AUC (0.9096 vs 0.8510), precision and MCC. The general result is that tree-based models dominate here, which follows from the structure of the data: the dominant signals are thresholds, and axis-aligned splits capture a threshold exactly while a linear or distance-based model can only approximate it.

Repository layout

```
.
├── app.py                    Streamlit app
├── churn_features.py         column definitions and the shared preprocessor
├── requirements.txt          pinned to the versions that trained the pickles
├── test_data.csv             the 667-row held-out split, labels included
├── data/                     the two source CSVs
└── model/
    ├── churn_modelling.ipynb exploration, training, evaluation
    ├── metrics_summary.csv   the comparison table above, as written by the notebook
    └── *.pkl                 six fitted pipelines
```

`churn_features.py` is imported by both the notebook and the app. That matters because a CSV round-trip loses dtypes - `pd.read_csv` reads `Area code` back as an integer and the plan columns back as text - and

the pipelines were fitted on the coerced versions. Putting `coerce_schema` in one module means the app cannot drift from the training code. The saved pipelines themselves are plain scikit-learn objects with no custom callables inside, so unpickling them does not depend on this module being importable.

Running it locally

```
git clone https://github.com/2025ac05989-bits/Bits-AIMLCZG565-Semester-1-Assignment-2.git
cd Bits-AIMLCZG565-Semester-1-Assignment-2
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

Then upload `test_data.csv` when the app asks for a CSV.

To retrain from scratch, run `model/churn_modelling.ipynb` top to bottom - it rewrites the six `.pkl` files, `metrics_summary.csv` and `test_data.csv`.

What the app does

- Upload a test CSV; the app validates that every required column is present and reports a clear message naming any that are missing.
- Pick one model from the dropdown, or tick "Compare all models" for a single table of all six scored on the uploaded file.
- Six evaluation metrics per model - accuracy, AUC, precision, recall, F1, MCC.
- A labelled confusion matrix and the full classification report, with a note on how many churners were missed versus how many loyal customers were flagged by mistake.
- A churn decision threshold slider. AUC does not move when it changes, since it is computed across all thresholds at once; the other five metrics do, which makes the precision/recall trade-off visible rather than theoretical.
- A ranked list of the highest-risk customers - the list a retention team would actually work from.

Built and tested on Python 3.13 (`.python-version` pins this for Streamlit Community Cloud).